

Worksheet No. - 7

Student Name: Ankush kumar

Branch: MCA

Semester: 2nd

Subject Name: TECHNICAL TRAINING

UID: 25MCA20305

Section/Group: 1A

Date of Performance: 31th Mar, 2026

Subject Code: 25CAP-652

Aim/Overview of the practical:

Implementation of JOINS in PostgreSQL

Objective:

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

1. Write queries to list students with their enrolled courses (INNER JOIN).
2. Find students not enrolled in any course (LEFT JOIN).
3. Display all courses with or without enrolled students (RIGHT JOIN).
4. Show students with department info using SELF JOIN or multiple joins.
5. Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

Input/Apparatus Used:

- PostgreSQL
- pgAdmin

Procedure/Algorithm/Code :

```
-- Departments

CREATE TABLE departments (
    id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

-- Students

CREATE TABLE students (
    id INT PRIMARY KEY,
```

```
name VARCHAR(50),

dept_id INT,

FOREIGN KEY (dept_id) REFERENCES departments(id)

);

-- Courses

CREATE TABLE courses (

    id INT PRIMARY KEY,

    course_name VARCHAR(50),

    dept_id INT,

    FOREIGN KEY (dept_id) REFERENCES departments(id)

);

-- Enrollments

CREATE TABLE enrollments (

    id INT PRIMARY KEY,

    student_id INT,

    course_id INT,

    FOREIGN KEY (student_id) REFERENCES students(id),

    FOREIGN KEY (course_id) REFERENCES courses(id)

);

INSERT INTO departments (id, dept_name) VALUES

(1, 'Computer Science'),

(2, 'Mechanical'),

(3, 'Electrical');
```

```
INSERT INTO students (id, name, dept_id) VALUES
(1, 'Ankush', 1),
(2, 'Rahul', 2),
(3, 'Priya', 1),
(4, 'Sneha', 3),
(5, 'Amit', 2),
(6, 'Karan', 1); -- Not enrolled (important for LEFT JOIN questions)
```

```
INSERT INTO courses (id, course_name, dept_id) VALUES
(1, 'Data Structures', 1),
(2, 'Operating Systems', 1),
(3, 'Thermodynamics', 2),
(4, 'Machine Design', 2),
(5, 'Circuits', 3);
```

```
INSERT INTO enrollments (id, student_id, course_id) VALUES
(1, 1, 1), -- Ankush → DS
(2, 1, 2), -- Ankush → OS
(3, 2, 3), -- Rahul → Thermodynamics
(4, 3, 1), -- Priya → DS
(5, 4, 5), -- Sneha → Circuits
(6, 5, 3), -- Amit → Thermodynamics
(7, 5, 4); -- Amit → Machine Design
```

--Question 1. Students with their enrolled courses (INNER JOIN)

```
SELECT s.name, c.course_name
```

FROM students s

INNER JOIN enrollments e ON s.id = e.student_id

INNER JOIN courses c ON e.course_id = c.id;

--Question 2. Students NOT enrolled in any course (LEFT JOIN)

SELECT s.id, s.name

FROM students s

LEFT JOIN enrollments e ON s.id = e.student_id

WHERE e.student_id IS NULL;

--Question 3. All courses with or without students (RIGHT JOIN)

SELECT c.course_name, s.name

FROM students s

RIGHT JOIN enrollments e ON s.id = e.student_id

RIGHT JOIN courses c ON e.course_id = c.id;

--Question 4. Students with department info (Multiple JOIN)

SELECT s.name, d.dept_name

FROM students s

JOIN departments d ON s.dept_id = d.id;

--Question 5. All possible student-course combinations (CROSS JOIN)

SELECT s.name, c.course_name

FROM students s

CROSS JOIN courses c;

Output:

	name character varying (20) 🔒	course_name character varying (20) 🔒
1	Ankush	Data Structures
2	Ankush	Operating Systems
3	Rahul	Thermodynamics
4	Priya	Data Structures
5	Sneha	Circuits
6	Amit	Thermodynamics
7	Amit	Machine Design

	id [PK] integer ✎	name character varying (20) ✎
1	6	John doe

	course_name character varying (20) 🔒	name character varying (20) 🔒
1	Data Structures	Ankush
2	Operating Systems	Ankush
3	Thermodynamics	Rahul
4	Data Structures	Priya
5	Circuits	Sneha
6	Thermodynamics	Amit
7	Machine Design	Amit

	name character varying (20) 🔒	dept_name character varying (20) 🔒
1	Ankush	Computer Science
2	Rahul	Mechanical
3	Priya	Computer Science
4	Sneha	Electrical
5	Amit	Mechanical
6	John doe	Electrical

	name character varying (20) 🔒	course_name character varying (20) 🔒
1	Ankush	Data Structures
2	Ankush	Operating Systems
3	Ankush	Thermodynamics
4	Ankush	Machine Design
5	Ankush	Circuits
6	Rahul	Data Structures
7	Rahul	Operating Systems
8	Rahul	Thermodynamics
9	Rahul	Machine Design
10	Rahul	Circuits
11	Priya	Data Structures
12	Priya	Operating Systems
13	Priya	Thermodynamics
14	Priya	Machine Design
15	Priya	Circuits
16	Sneha	Data Structures
17	Sneha	Operating Systems
18	Sneha	Thermodynamics
19	Sneha	Machine Design
20	Sneha	Circuits
21	Amit	Data Structures

Learning outcomes (What I have learnt):

1. I learned how to use INNER JOIN to display only matching records between students and courses.
2. I understood that LEFT JOIN combined with IS NULL helps me find records that do not have corresponding entries.
3. I realized that RIGHT JOIN (or equivalent LEFT JOIN) allows me to include all records from one table even if there is no match in another.
4. I learned how to combine multiple tables using JOIN to retrieve related information like student names and their departments.
5. I understood that CROSS JOIN generates all possible combinations between two tables, resulting in a Cartesian product.